

# CS121 Data Structures

## Linked Lists

Apig Aramian  
aaramian@aua.am



Fall 2026

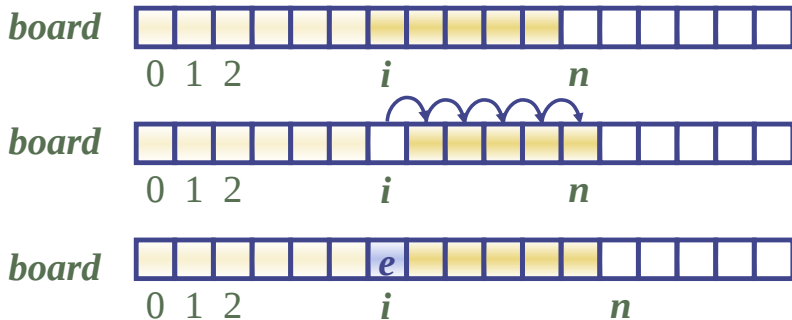
# Important Data and Statistics



- ▶ 24 classes remaining
- ▶ 32 days till the Midterm exam I

## Adding an Array Entry

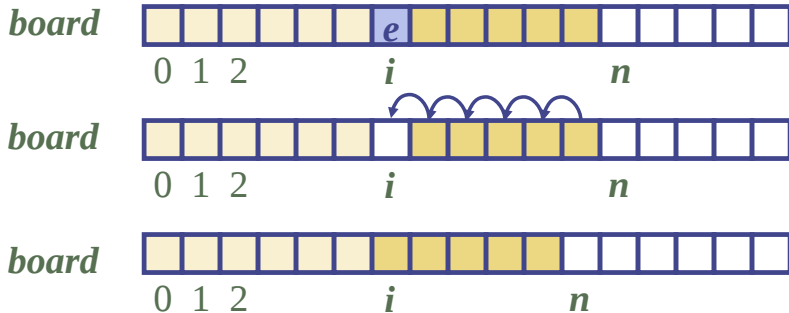
To add an entry  $e$  into array  $board$  at index  $i$ , we need to make room for it by shifting forward the  $n - i$  entries  $board[i], \dots, board[n - 1]$



## Removing an Array Entry

To remove the entry  $e$  at index  $i$ , we need to fill the hole left by  $e$  by shifting backward the  $n - i - 1$  elements

$board[i + 1], \dots, board[n - 1]$



# Singly Linked Lists

Drawbacks of *array* as an ordered data structure:

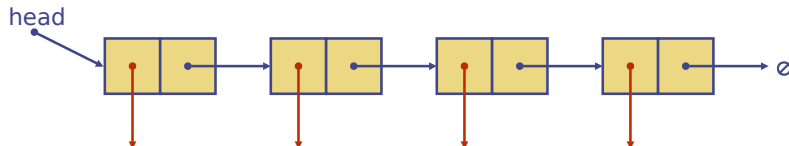
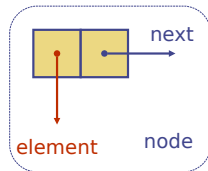
- ▶ fixed capacity
- ▶ expensive insertions and deletions at interior positions (shifting many elements)

**Linked list** provides an alternative to an array-based structure.

A linked list is a collection of **nodes** that collectively form a linear sequence.

In a **singly linked list**, each node stores:

- ▶ a reference to an object that is an element of the sequence
- ▶ a reference to the next node of the list



# Linked List Terms

The linked list instance must keep a reference to the first node of the list, known as the **head**.

The last node of the list is known as the **tail**.

**Traversing** the linked list—starting at the head and moving from one node to another by following each node's `next` reference.

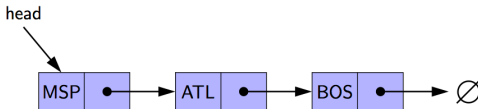
The tail has **null** as its `next` reference.

Commonly, a reference to the tail node is also stored, as is the count of the total number of nodes in the list (its **size**).

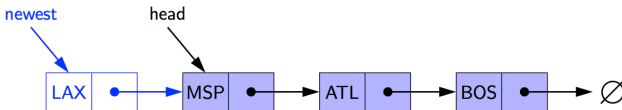
# Inserting at the Head

**Algorithm** addFirst(e):

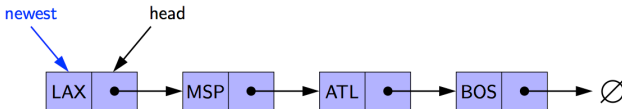
```
newest = Node(e)    {create new node instance storing reference to element e}  
newest.next = head  {set new node's next to reference the old head node}  
head = newest        {set variable head to reference the new node}  
size = size + 1     {increment the node count}
```



(a)



(b)

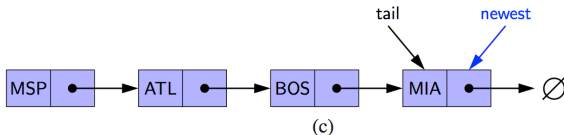
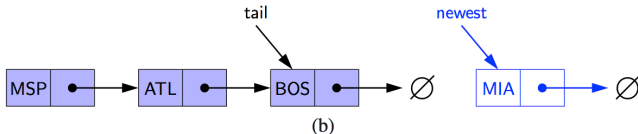
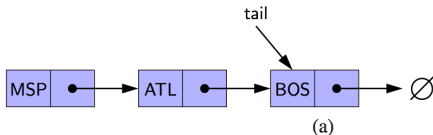


(c)

# Inserting at the Tail

**Algorithm** addLast(e):

```
newest = Node(e)      {create new node instance storing reference to element e}  
newest.next = null    {set new node's next to reference the null object}  
tail.next = newest     {make old tail node point to new node}  
tail = newest          {set variable tail to reference the new node}  
size = size + 1       {increment the node count}
```



# Removing from the Head

**Algorithm** removeFirst:

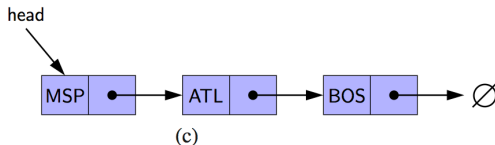
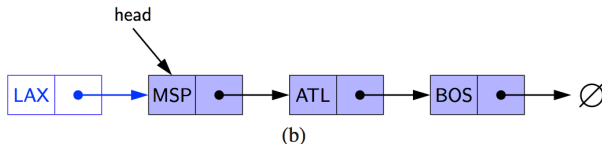
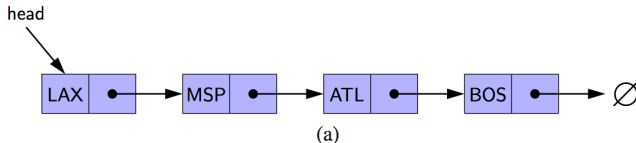
if head == null then  
the list is empty.

head = head.next

size = size - 1

{make head point to next node (or null)}

{decrement the node count}



# Removing from the Tail

Removing from the tail of a singly linked list is not efficient!

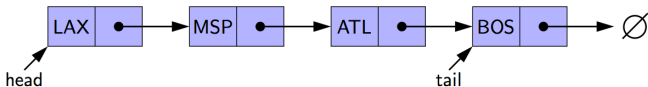
Why?

# Removing from the Tail

Removing from the tail of a singly linked list is not efficient!

Why?

There is no constant-time way to update the tail to point to the previous node



# Interface of a Singly Linked List

`size()` Returns the number of elements in the list.

`isEmpty()` Returns **true** if the list is empty, and **false** otherwise.

`first()` Returns (but does not remove) the first element in the list.

`last()` Returns (but does not remove) the last element in the list.

`addFirst(e)` Adds a new element to the front of the list.

`addLast(e)` Adds a new element to the end of the list.

`removeFirst()` Removes and returns the first element of the list.

# Singly Linked List Implementation: Node

```
1 public class SinglyLinkedList<E> {  
2     //----- nested Node class -----  
3     private static class Node<E> {  
4         private E element;           // reference to the element stored at this node  
5         private Node<E> next;       // reference to the subsequent node in the list  
6         public Node(E e, Node<E> n) {  
7             element = e;  
8             next = n;  
9         }  
10        public E getElement() { return element; }  
11        public Node<E> getNext() { return next; }  
12        public void setNext(Node<E> n) { next = n; }  
13    } //----- end of nested Node class -----
```

# Singly Linked List Implementation I

```
1  public class SinglyLinkedList<E> {  
...    (nested Node class goes here)  
14  // instance variables of the SinglyLinkedList  
15  private Node<E> head = null;      // head node of the list (or null if empty)  
16  private Node<E> tail = null;     // last node of the list (or null if empty)  
17  private int size = 0;              // number of nodes in the list  
18  public SinglyLinkedList() { }      // constructs an initially empty list  
19  // access methods  
20  public int size() { return size; }  
21  public boolean isEmpty() { return size == 0; }  
22  public E first() {                  // returns (but does not remove) the first element  
23      if (isEmpty()) return null;  
24      return head.getElement();  
25  }  
26  public E last() {                  // returns (but does not remove) the last element  
27      if (isEmpty()) return null;  
28      return tail.getElement();  
29  }
```

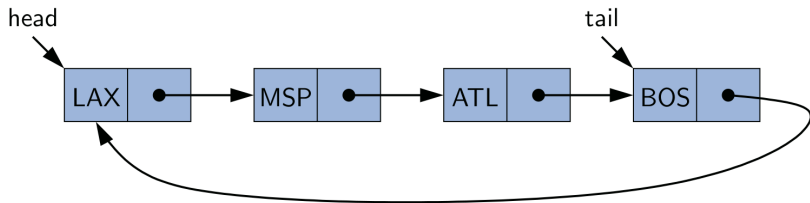
# Singly Linked List Implementation II

```
30 // update methods
31 public void addFirst(E e) { // adds element e to the front of the list
32     head = new Node<>(e, head); // create and link a new node
33     if (size == 0)
34         tail = head; // special case: new node becomes tail also
35     size++;
36 }
37 public void addLast(E e) { // adds element e to the end of the list
38     Node<E> newest = new Node<>(e, null); // node will eventually be the tail
39     if (isEmpty())
40         head = newest; // special case: previously empty list
41     else
42         tail.setNext(newest); // new node after existing tail
43     tail = newest; // new node becomes the tail
44     size++;
45 }
46 public E removeFirst() { // removes and returns the first element
47     if (isEmpty()) return null; // nothing to remove
48     E answer = head.getElement();
49     head = head.getNext(); // will become null if list had only one node
50     size--;
51     if (size == 0)
52         tail = null; // special case as list is now empty
53     return answer;
54 }
55 }
```

# Circularly Linked Lists

There are applications in which data can be viewed as having a **cyclic order**, with well-defined neighbouring relationships, but no fixed beginning or end.

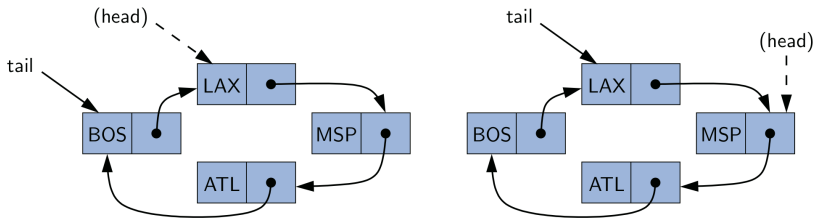
A **circularly linked list** is a singularly linked list in which the next reference of the tail node is set to refer back to the head of the list (rather than **null**).



# Rotating

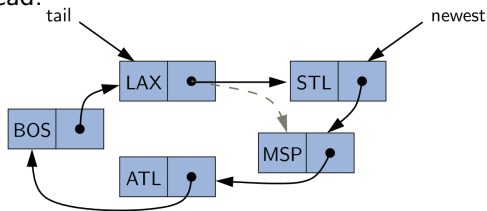
We no longer explicitly maintain the head reference. Thus we save a bit on memory usage and make the code simpler and more efficient.

When **rotating** the linked list, we simply advance the tail reference to point to the node that follows it.



# Adding at the Head/Tail

Add at the head:



Add at the tail: add at the head and immediately rotate

# Circularly Linked List Implementation I

```
1  public class CircularlyLinkedList<E> {  
...    (nested node class identical to that of the SinglyLinkedList class)  
14  // instance variables of the CircularlyLinkedList  
15  private Node<E> tail = null;           // we store tail (but not head)  
16  private int size = 0;                   // number of nodes in the list  
17  public CircularlyLinkedList() { }       // constructs an initially empty list  
18  // access methods  
19  public int size() { return size; }  
20  public boolean isEmpty() { return size == 0; }  
21  public E first() {                      // returns (but does not remove) the first element  
22      if (isEmpty()) return null;  
23      return tail.getNext().getElement(); // the head is *after* the tail  
24  }  
25  public E last() {                       // returns (but does not remove) the last element  
26      if (isEmpty()) return null;  
27      return tail.getElement();  
28  }
```

# Circularly Linked List Implementation II

```
29  // update methods
30  public void rotate() {           // rotate the first element to the back of the list
31      if (tail != null)           // if empty, do nothing
32          tail = tail.getNext();   // the old head becomes the new tail
33  }
34  public void addFirst(E e) {       // adds element e to the front of the list
35      if (size == 0) {
36          tail = new Node<>(e, null);
37          tail.setNext(tail);       // link to itself circularly
38      } else {
39          Node<E> newest = new Node<>(e, tail.getNext());
40          tail.setNext(newest);
41      }
42      size++;
43  }
44  public void addLast(E e) {        // adds element e to the end of the list
45      addFirst(e);                 // insert new element at front of list
46      tail = tail.getNext();       // now new element becomes the tail
47  }
48  public E removeFirst() {          // removes and returns the first element
49      if (isEmpty()) return null;   // nothing to remove
50      Node<E> head = tail.getNext();
51      if (head == tail) tail = null; // must be the only node left
52      else tail.setNext(head.getNext()); // removes "head" from the list
53      size--;
54      return head.getElement();
55  }
56 }
```

# Doubly Linked Lists

Limitations of *singly linked list*:

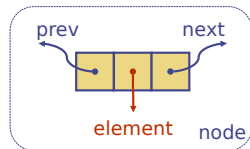
- ▶ unable to efficiently delete a node at the tail
- ▶ cannot efficiently delete a node from an interior position if given a reference to it (cannot determine the preceding node)

In a **doubly linked list** each node keeps an explicit reference to the node before it and a reference to the node after it.

A doubly linked list can be traversed forward and backward.

In a **doubly linked list**, each node stores:

- ▶ a reference to an object that is an element of the sequence
- ▶ a reference to the previous node of the list
- ▶ a reference to the next node of the list

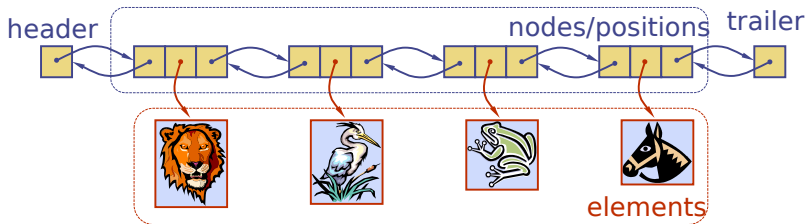


# Header and Trailer Sentinels

It helps to add special nodes at both ends of the list:

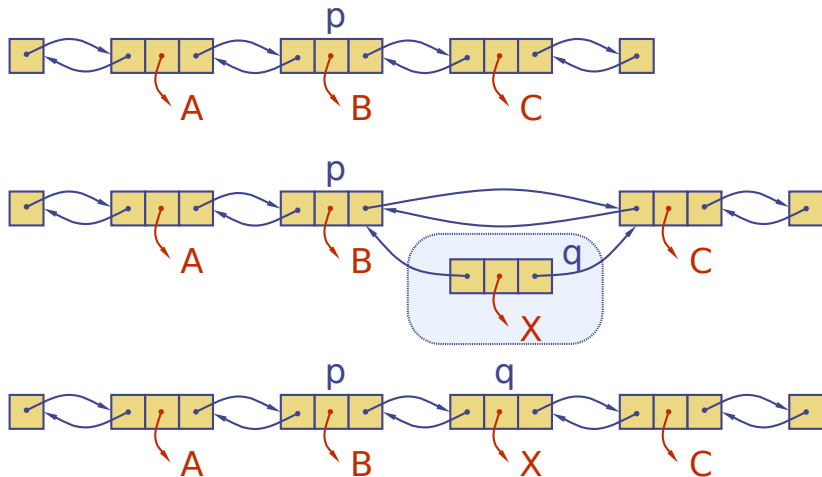
- ▶ a **header** node at the beginning of the list
- ▶ a **trailer** node at the end of the list

These 'dummy' nodes are known as **sentinels** (or guards). They do not store elements.



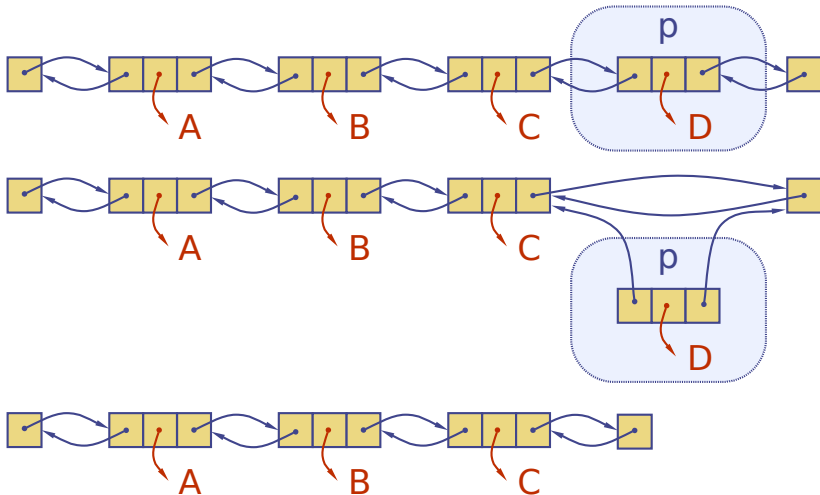
# Insertion

Every insertion takes place between a pair of existing nodes.



# Deletion

Every deletion takes place between a pair of existing nodes.



# Interface of a Doubly Linked List

`size()` Returns the number of elements in the list.

`isEmpty()` Returns **true** if the list is empty, and **false** otherwise.

`first()` Returns (but does not remove) the first element in the list.

`last()` Returns (but does not remove) the last element in the list.

`addFirst(e)` Adds a new element to the front of the list.

`addLast(e)` Adds a new element to the end of the list.

`removeFirst()` Removes and returns the first element of the list.

`removeLast()` Removes and returns the last element of the list.

# Doubly Linked List Implementation: Node

```
1  /** A basic doubly linked list implementation. */
2  public class DoublyLinkedList<E> {
3      //----- nested Node class -----
4      private static class Node<E> {
5          private E element;           // reference to the element stored at this node
6          private Node<E> prev;        // reference to the previous node in the list
7          private Node<E> next;        // reference to the subsequent node in the list
8          public Node(E e, Node<E> p, Node<E> n) {
9              element = e;
10             prev = p;
11             next = n;
12         }
13         public E getElement() { return element; }
14         public Node<E> getPrev() { return prev; }
15         public Node<E> getNext() { return next; }
16         public void setPrev(Node<E> p) { prev = p; }
17         public void setNext(Node<E> n) { next = n; }
18     } //----- end of nested Node class -----
19 }
```

# Doubly Linked List Implementation I

```
20 // instance variables of the DoublyLinkedList
21 private Node<E> header;           // header sentinel
22 private Node<E> trailer;          // trailer sentinel
23 private int size = 0;              // number of elements in the list
24 /** Constructs a new empty list. */
25 public DoublyLinkedList() {
26     header = new Node<>(null, null, null); // create header
27     trailer = new Node<>(null, header, null); // trailer is preceded by header
28     header.setNext(trailer); // header is followed by trailer
29 }
30 /** Returns the number of elements in the linked list. */
31 public int size() { return size; }
32 /** Tests whether the linked list is empty. */
33 public boolean isEmpty() { return size == 0; }
34 /** Returns (but does not remove) the first element of the list. */
35 public E first() {
36     if (isEmpty()) return null;
37     return header.getNext().getElement(); // first element is beyond header
38 }
39 /** Returns (but does not remove) the last element of the list. */
40 public E last() {
41     if (isEmpty()) return null;
42     return trailer.getPrev().getElement(); // last element is before trailer
43 }
```

# Doubly Linked List Implementation II

```
44 // public update methods
45 /** Adds element e to the front of the list. */
46 public void addFirst(E e) {
47     addBetween(e, header, header.getNext()); // place just after the header
48 }
49 /** Adds element e to the end of the list. */
50 public void addLast(E e) {
51     addBetween(e, trailer.getPrev(), trailer); // place just before the trailer
52 }
53 /** Removes and returns the first element of the list. */
54 public E removeFirst() {
55     if (isEmpty()) return null; // nothing to remove
56     return remove(header.getNext()); // first element is beyond header
57 }
58 /** Removes and returns the last element of the list. */
59 public E removeLast() {
60     if (isEmpty()) return null; // nothing to remove
61     return remove(trailer.getPrev()); // last element is before trailer
62 }
63
```

# Doubly Linked List Implementation III

```
64 // private update methods
65 /** Adds element e to the linked list in between the given nodes. */
66 private void addBetween(E e, Node<E> predecessor, Node<E> successor) {
67     // create and link a new node
68     Node<E> newest = new Node<>(e, predecessor, successor);
69     predecessor.setNext(newest);
70     successor.setPrev(newest);
71     size++;
72 }
73 /** Removes the given node from the list and returns its element. */
74 private E remove(Node<E> node) {
75     Node<E> predecessor = node.getPrev();
76     Node<E> successor = node.getNext();
77     predecessor.setNext(successor);
78     successor.setPrev(predecessor);
79     size--;
80     return node.getElement();
81 }
82 } //----- end of DoublyLinkedList class -----
```

# Summary

## Reading

Sections 3.2–3.6 of the main textbook

## Questions?